

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Americo Ignacio Acuña Rodríguez

8 de septiembre de 2026

16:52

1. Introducción

El análisis asintótico tradicional clasifica algoritmos según su crecimiento teórico, omitiendo restricciones de hardware como la jerarquía de memorias o las optimizaciones del compilador. Este estudio evalúa empíricamente dos problemas: el ordenamiento unidimensional (MergeSort, QuickSort, PatienceSort y `std::sort`) y la multiplicación de matrices cuadradas (Naive vs Strassen), contrastando la teoría formal frente al rendimiento práctico. El código fuente y los datos están disponibles en: https://github.com/Ameripro2006/INF221_T1_2026-2.

2. Entorno experimental

Los algoritmos (desarrollados en C++17 y compilados con `g++ -O2`) se ejecutaron en un entorno Linux (Kernel 6.x) equipado con un procesador AMD Ryzen 5 7600X y 32 GB de RAM DDR5. Los tiempos de ejecución se midieron utilizando `std::chrono::high_resolution_clock`, y el consumo de memoria máxima (Peak RSS) se registró mediante la llamada al sistema `getrusage`.

3. Resultados y Análisis

3.1. Algoritmos de Ordenamiento

El análisis experimental expone una fuerte sensibilidad a la distribución inicial de los datos.

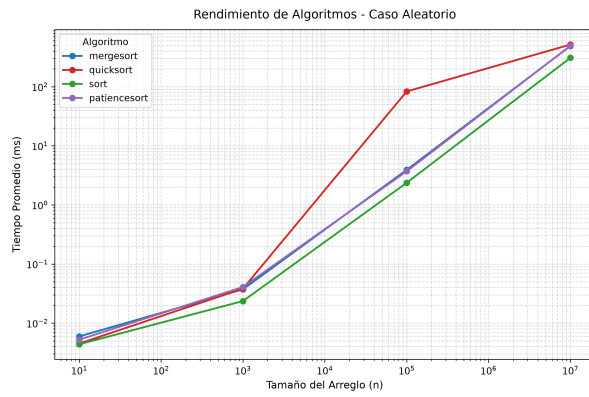


Figura 1: Tiempo: Caso aleatorio.

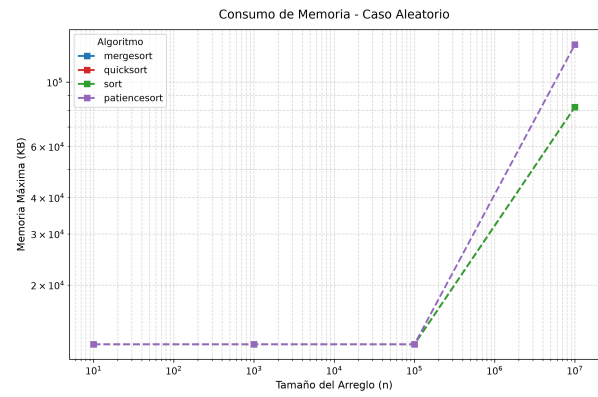


Figura 2: Memoria: Caso aleatorio.

En la Figura 1, `std::sort` logra la menor latencia. QuickSort sufre una degradación catastrófica a $\mathcal{O}(N^2)$ con arreglos ordenados, consumiendo toda la pila (*Stack Overflow*) o excediendo el límite de tiempo para $N \geq 10^6$. MergeSort conserva su tasa teórica $\mathcal{O}(N \log N)$, mientras que PatienceSort mantiene su cota pero con una alta constante multiplicativa. Respecto a la memoria (Figura 2), `std::sort` domina operando *in-place*, mientras MergeSort y PatienceSort exhiben un crecimiento $\mathcal{O}(N)$ por sus estructuras auxiliares.

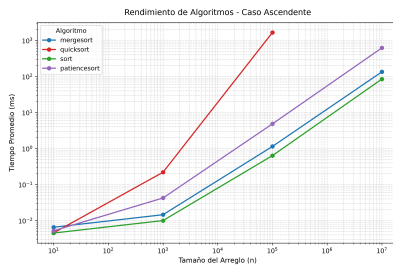


Figura 3: Ascendente.

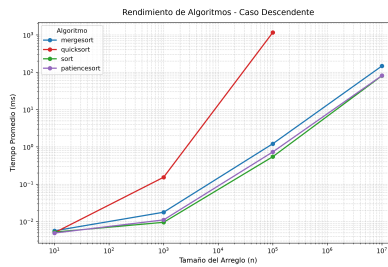
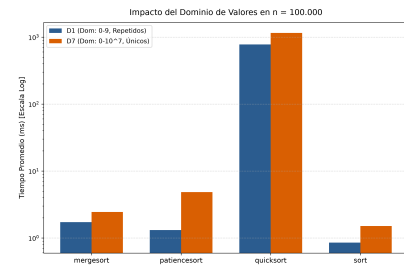


Figura 4: Descendente.

Figura 5: Dominio (10^5).

3.2. Multiplicación de Matrices

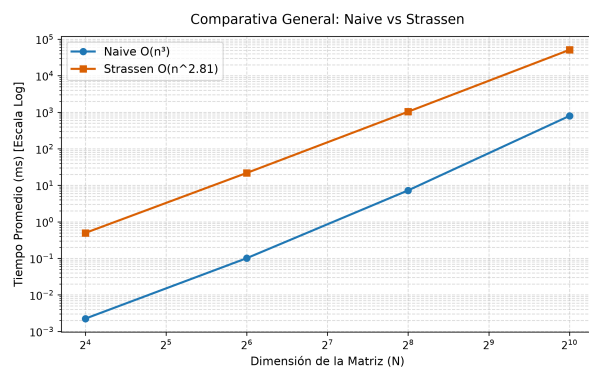


Figura 6: Tiempo: Naive vs Strassen.

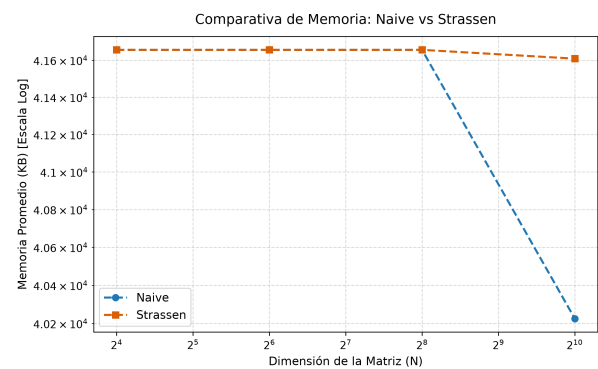


Figura 7: Memoria: Naive vs Strassen.

Naive supera por órdenes de magnitud a Strassen (Figura 6). La recursividad profunda en $N = 1024$ genera $\approx 3,29 \times 10^8$ llamadas, asfixiando la ganancia asintótica por el inmenso *overhead*. Esto se refleja en la Figura 7: Strassen exige una asignación masiva en el *heap*, mientras Naive mantiene una huella estable $\mathcal{O}(N^2)$. El tiempo demostró ser agnóstico a la estructura y dominio de las matrices, ya que ninguna implementación omite cálculos sobre ceros.

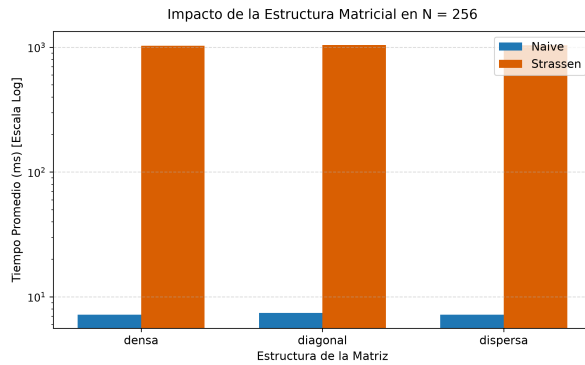


Figura 8: Estructura interna.

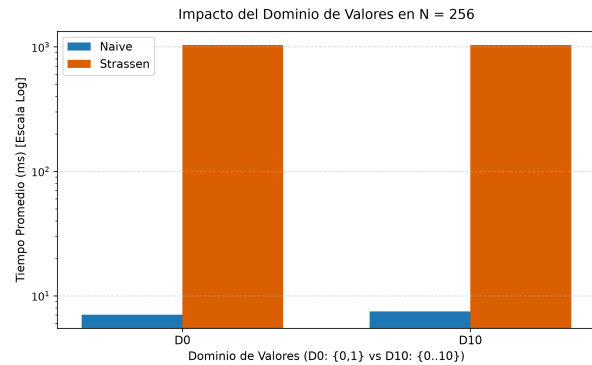


Figura 9: Dominio numérico.

4. Conclusiones

La notación asintótica teórica resulta insuficiente en entornos reales sin considerar la arquitectura. En ordenamiento, la topología inicial condiciona fatalmente la partición ingenua en QuickSort (causando *Stack Overflow*) y el número de pilas en PatienceSort, destacando la robustez de MergeSort y `std::sort` (*in-place*). En matrices, la ventaja teórica de Strassen ($\mathcal{O}(N^{2,81})$) es anulada en $N \leq 1024$ por el colosal costo de gestión dinámica de memoria y la degradación de localidad frente al bucle contiguo de Naive, fuertemente optimizado por el compilador.